PRACTICAL DEVOPS PROJECT GUIDE

Expense Management Application

Backend → MySQL → Nginx → DNS → Application Load Balancer

A step-by-step AWS deployment workbook with real-world explanations, commands, architecture, troubleshooting and a technical glossary.

Designed for hands-on practice on Amazon Linux / AWS EC2

How to use this guide

This is a practical learning document. Read a concept, understand the real-world example, perform the lab step, and verify the result before moving forward.

- Use a separate AWS lab account or an environment where you are authorised to create resources.
- Use placeholders such as <BACKEND_PRIVATE_IP>, <DOMAIN> and <DB_SECRET>. Replace them with your actual values.
- Never paste passwords, private keys, access tokens or database secrets into notes, screenshots, Git repositories or chat messages.
- AWS resources can cost money. Check current pricing, set a budget alert, and delete lab resources when finished.

Important — The examples are reference configurations, not a claim that your exact application has the same API routes, schema, file paths or Node.js version. Inspect your project files and adapt the commands.

Contents and learning route

- 1. Understand the project and client-server architecture
- 2. Networking foundations: IP, ports, protocols, packets and security groups
- 3. API, REST, HTTP methods, status codes and Postman
- 4. Backend application: Node.js, npm, dependencies and environment variables
- 5. Linux service management with systemd
- 6. MySQL database: server, schema, tables, users and connectivity
- 7. Frontend, web server and Nginx
- 8. Forward proxy, reverse proxy and load balancing
- 9. DNS from first principles: recursive resolver, root, TLD, authoritative DNS, records and TTL
- 10. Route 53 domain setup and records
- 11. AWS Application Load Balancer: listeners, rules, target groups and health checks
- 12. End-to-end deployment runbook
- 13. Testing, troubleshooting, security and operations
- 14. Large glossary: technical words explained with real-world examples
- 15. Practical exercises and interview questions

1. Project overview: what are we building?

We are deploying an Expense Management web application. A user opens a website, signs in, views expenses, creates a new expense and saves it. The browser displays the frontend. The frontend calls backend APIs. The backend validates the request and reads or writes data in MySQL.

1.1 The architecture

Layer	Lab component	Responsibility	Real-world analogy
User/client	Browser or mobile app	Starts requests and displays responses	Customer using a bank's mobile app
DNS	Route 53	Finds the destination associated with a domain name	Contact directory that helps find the right service
Entry point	Application Load Balancer	Accepts web traffic and routes it to healthy targets	Reception desk directing visitors to an available counter
Frontend	Two EC2 instances with Nginx	Serves HTML, CSS, JavaScript; may proxy API calls	Shopfront/customer-facing counter
Backend	EC2 with Node.js/Express	Runs business logic and exposes APIs	Staff member processing a customer's request
Database	EC2 with MySQL	Stores persistent application data	Secure filing room / ledger

1.2 Request flow in plain English

1. The user types https://<DOMAIN> in a browser.
2. The browser asks DNS where <DOMAIN> should go. DNS returns the address/alias for the public entry point, typically the ALB.
3. The browser establishes a TCP connection and, for HTTPS, negotiates TLS encryption with the ALB.
4. The ALB listener receives the request. Listener rules choose a target group.
5. The ALB selects a healthy frontend EC2 target and forwards the request.
6. Nginx on the frontend returns static website files. When the browser calls an API, Nginx can proxy that request to the backend.
7. The Node.js backend receives the API request, checks authentication and input, and runs application logic.
8. The backend connects to MySQL over the private network, executes a query and receives the result.
9. The backend returns an HTTP response. Nginx and the ALB pass it back to the browser.
10. The browser updates the screen with the result.

Key idea — The browser does not normally connect directly to MySQL. The backend is the trusted application layer between the user and the database.

2. Client-server architecture and networking basics

2.1 What is a client?

A client is software or a device that requests a service. A browser, mobile app, Postman, curl command or another application can act as a client.

Real example: When you open your bank's website on your phone, the browser is the client. It requests the login page and later sends login/API requests.

2.2 What is a server?

A server is a computer or software process that listens for requests and provides a service. One EC2 virtual machine can run multiple server processes, such as Nginx and Node.js, each listening on different ports.

Real example: A bank's application server receives a fund-transfer request, checks rules and communicates with databases or other services.

2.3 Client-server architecture

Client-server architecture is a model in which clients request services and servers provide them over a network. The client and server may run on different machines, in different networks or in different regions.

Example	Client	Server
Open a website	Chrome browser	Nginx/web server
Check an expense	Frontend JavaScript	Node.js API server
Run SQL query	Node.js database driver	MySQL server
Send API request	Postman	Backend API
Resolve a domain	Laptop/phone DNS stub resolver	Recursive DNS resolver and authoritative DNS servers

2.4 IP address

An IP address identifies a network interface so that packets can be routed to it. IPv4 looks like 192.0.2.10. IPv6 is longer, for example 2001:db8::10. An EC2 instance can have a private IP and, depending on its configuration, a public IPv4 address.

Real example: Think of an IP address as the street address of a building. Network packets need a destination address to reach the correct machine.

2.5 Public IP vs private IP

Type	Meaning	Example in this project
Public IP	Routable on the public internet, subject to network controls	A public-facing ALB; possibly a bastion or lab host
Private IP	Used for communication within private networks connected by routing	Backend and MySQL EC2 private addresses

Production design — Keep the database in private subnets and do not expose TCP 3306 to the internet. Backend-to-database traffic should use private addressing.

2.6 Port

A port is a numbered endpoint used by a transport protocol on a host. An IP identifies the machine/interface; a port identifies the service/application endpoint.

Port	Common use	Project example
22/TCP	SSH	Administrator connects to Linux, preferably from a trusted source or via SSM
80/TCP	HTTP	ALB to Nginx frontend
443/TCP	HTTPS	Browser to ALB
8080/TCP	Application-specific HTTP	Nginx frontend to Node.js backend, if configured
3306/TCP	MySQL	Backend to database

2.7 Protocol

A protocol is an agreed set of rules for communication. HTTP defines web request/response behaviour. TCP provides reliable ordered transport. DNS resolves names. TLS encrypts and authenticates connections.

2.8 TCP and UDP

TCP establishes a connection and provides reliable, ordered byte delivery. HTTP/1.1 and HTTP/2 commonly use TCP. UDP sends datagrams without TCP's connection/retransmission guarantees; DNS commonly uses UDP, with TCP also used when required.

Analogy: TCP is like a tracked courier that confirms delivery and preserves order. UDP is like sending individual postcards without the same delivery guarantees.

2.9 Firewall and AWS Security Group

A firewall controls which network traffic is allowed. An AWS Security Group is a stateful virtual firewall attached to resources such as EC2 instances and load balancers. Stateful means response traffic for an allowed connection is automatically permitted.

Resource security group	Inbound rule example	Why
ALB-SG	TCP 443 from intended internet clients	Users can reach HTTPS entry point
Frontend-SG	TCP 80 from ALB-SG only	Only the ALB can reach frontend web servers
Backend-SG	TCP 8080 from Frontend-SG only	Only frontend tier can call backend
DB-SG	TCP 3306 from Backend-SG only	Only backend tier can connect to MySQL
Admin access	TCP 22 from a trusted admin IP, or use SSM	Avoid opening SSH to everyone

Avoid — Do not use 0.0.0.0/0 for SSH, backend application ports or MySQL. Public access should normally be limited to the ALB listener.

3. API, REST, HTTP and Postman

3.1 What is an API?

API stands for Application Programming Interface. It is a defined way for one software component to request data or an operation from another component.

Real example: A food delivery app needs restaurant menus. Its frontend sends an API request to a backend. The backend returns menu data, often as JSON. The frontend displays that data.

In our project: the frontend asks the backend, 'Show the expenses for this user.' The backend checks access, reads MySQL and returns the matching expenses.

3.2 API endpoint

An endpoint is a particular URL/path and method combination exposed by an API. For example, GET /api/expenses may list expenses and POST /api/expenses may create one. The exact endpoints depend on the code.

3.3 Request and response

Part	Meaning	Example
Method	What operation is requested	GET
URL/path	Which resource or endpoint	/api/expenses/42
Headers	Metadata about the request	Authorization, Content-Type
Body	Optional data sent by client	JSON expense fields
Status code	Outcome category	200, 201, 400, 401, 500
Response body	Data returned by server	JSON expense record

3.4 JSON

JSON means JavaScript Object Notation. It is a text format commonly used to exchange structured data between APIs.

```
{
  "description": "Fuel",
  "amount": 1200,
  "category": "Travel"
}
```

The backend must validate JSON. A client can send missing fields, invalid values or malicious input; never trust the request simply because it is JSON.

3.5 REST API

REST is an architectural style for designing network APIs around resources. REST APIs commonly use HTTP methods and resource-oriented URLs. Not every HTTP API is RESTful.

Real example: /api/expenses represents a collection of expense resources. /api/expenses/42 identifies one expense.

Method	Meaning	CRUD	Real example
GET	Retrieve data	Read	View all expenses
POST	Create a resource or submit an operation	Create	Add a new expense
PUT	Replace a resource representation	Update	Replace expense 42 fields
PATCH	Partially modify a resource	Update	Change only amount or category
DELETE	Delete a resource	Delete	Remove expense 42

3.6 CRUD

CRUD means Create, Read, Update and Delete. These are the four basic data operations used in many applications.

3.7 Common HTTP status codes

Code	Meaning	Expense app example
200 OK	Request succeeded	Expense list returned
201 Created	New resource created	Expense saved
202 Accepted	Request accepted for later processing	Report generation job queued
204 No Content	Success, no response body	Expense deleted
400 Bad Request	Request is malformed or invalid	Amount missing or wrong type
401 Unauthorized	Authentication is missing or invalid	Login/token required
403 Forbidden	User is authenticated but not allowed	User tries to access another user's expense
404 Not Found	Resource or route not found	Expense ID 999 does not exist
409 Conflict	Request conflicts with current state	Duplicate operation
422 Unprocessable Content	Request understood but validation fails	Amount is negative when not allowed
429 Too Many Requests	Rate limit exceeded	Too many requests in short period
500 Internal Server Error	Unexpected server error	Unhandled backend exception
502 Bad Gateway	Proxy received an invalid response from upstream	Nginx cannot get a valid backend response
503 Service Unavailable	Service temporarily unavailable	No healthy ALB targets
504 Gateway Timeout	Upstream did not respond in time	Backend or database response is too slow

3.8 Authentication vs authorization

Authentication answers: 'Who are you?' Authorization answers: 'What are you allowed to do?' A successful login does not mean the user can read every other user's expenses.

3.9 What is Postman?

Postman is an API client used to send HTTP requests and inspect responses. It is useful for testing a backend before the frontend is ready.

11. Set the request method to GET and URL to http://<BACKEND_PRIVATE_IP>:8080/health, if the backend has that route.
12. Send a POST request to the documented create-expense endpoint.
13. Choose Body → raw → JSON and enter the fields expected by your API.
14. Add the required Authorization header if the API uses authentication.
15. Check status code, response body, headers and response time.

Important — Do not assume /health or /api/expenses exists. Inspect the backend source code, README or route files to confirm the actual paths.

4. Backend application: Node.js, npm and environment

4.1 What is a backend?

The backend is the server-side part of an application. It receives requests, applies business rules, authenticates and authorizes users, communicates with databases or external services, and sends responses.

Real example: When a user submits a ₹500 travel expense, the backend checks that the user is logged in, validates the amount and category, saves the record in MySQL and returns success.

4.2 Node.js

Node.js is a JavaScript runtime that lets JavaScript execute outside a browser, commonly for backend services, APIs and command-line tools. It is not the same thing as a web framework.

4.3 Express.js

Express is a web framework for Node.js. It provides routing and middleware patterns for building HTTP servers and APIs.

4.4 npm, package.json and package-lock.json

Term	Meaning	Real-world example
npm	Node Package Manager / npm CLI	Tool used to install project libraries
package.json	Project metadata, scripts and dependency declarations	Shopping list of libraries and project commands
package-lock.json	Records resolved dependency tree and versions	Exact shopping list/version record for repeatable installs
node_modules	Directory containing installed packages	Downloaded library shelf for the project
Dependency	External package the app uses	Express, mysql2, dotenv
Runtime	Environment that executes a program	Node.js runs index.js

4.5 Inspect the project before deploying

```
cd /app

pwd

ls -la

cat package.json

test -f package-lock.json && echo "Lock file exists"

find . -maxdepth 2 -type f | sort | head -100

node --version

npm --version
```

Check the project's documented Node.js version, start script, entry-point file, environment variable names, database driver and schema instructions. Do not guess that the entry point is index.js.

4.6 Install dependencies

```
cd /app

npm ci --omit=dev
```

npm ci installs the exact dependency versions recorded in package-lock.json and is intended for repeatable deployments. It generally requires a lock file consistent with package.json. If the project has no lock file, resolve that as a project build issue rather than silently changing production dependencies.

4.7 Environment variables

Environment variables are configuration values supplied to a process from its environment. They help keep environment-specific settings out of source code.

```
NODE_ENV=production

PORT=8080

DB_HOST=<DB_PRIVATE_DNS_OR_IP>

DB_USER=<APP_DB_USER>

DB_PASSWORD=<SECRET>

DB_NAME=transactions
```

The backend reads these values at runtime. The exact names are application-specific. If code expects DB_PWD but the environment file defines DB_PASSWORD, the connection may fail.

4.8 Why not hard-code secrets?

A password embedded in source code can be copied into Git, backups, logs or screenshots. Use AWS Secrets Manager or Systems Manager Parameter Store where appropriate, or a tightly permissioned root-owned environment file for a small lab.

4.9 Run backend temporarily for a test

```
cd /app
```

```
# Use the application's documented start command.
# Example only:
PORT=8080 node index.js
```

This foreground test is useful for seeing startup errors. Stop with Ctrl+C. Do not use a foreground SSH session as the permanent production service.

4.10 Verify listening port

```
sudo ss -lntp
curl -i http://127.0.0.1:8080/health
```

ss displays listening sockets and their processes. curl sends an HTTP request. Replace the health path with a route that actually exists.

5. Linux systemd: keep backend running

5.1 What is a process?

A process is a running instance of a program. When Node.js starts your backend, Linux creates a process with a process ID (PID).

5.2 What is a daemon/service?

A daemon is a background process that provides a service. A service is a managed unit of work, such as the backend API or Nginx.

5.3 What is systemd?

systemd is the service manager used by many modern Linux distributions. It can start services during boot, stop/restart them, apply restart policies and integrate logs with the journal.

5.4 Create a dedicated service account

```
sudo useradd --system --home-dir /app --shell /sbin/nologin expense
sudo chown -R expense:expense /app
```

A dedicated service account limits the damage if the application is compromised. Confirm ownership and file permissions so the service can read application files and write only where needed.

5.5 Example backend.service

```
sudo vi /etc/systemd/system/backend.service

[Unit]
Description=Expense Management Backend
Wants=network-online.target
After=network-online.target

[Service]
Type=simple
User=expense
Group=expense
WorkingDirectory=/app
Environment=NODE_ENV=production
EnvironmentFile=/etc/expense/backend.env
ExecStart=/usr/bin/node /app/index.js
```

```
Restart=on-failure
RestartSec=5
NoNewPrivileges=true
PrivateTmp=true

[Install]
WantedBy=multi-user.target
```

Adjust /usr/bin/node and /app/index.js to the real paths. A systemd unit is configuration, not application code.

5.6 Create a protected environment file

```
sudo install -d -o root -g expense -m 0750 /etc/expense
sudo vi /etc/expense/backend.env
sudo chown root:expense /etc/expense/backend.env
sudo chmod 0640 /etc/expense/backend.env
```

Add the application's exact environment variable names. Avoid echoing secrets into shell history. A secrets manager is preferable for production.

5.7 Enable and start service

```
sudo systemctl daemon-reload
sudo systemctl enable --now backend
sudo systemctl status backend
sudo journalctl -u backend -n 100 --no-pager
sudo journalctl -u backend -f
```

Command	Meaning
daemon-reload	Read changed or newly created systemd unit files
enable	Configure service to start at boot
start	Start service now
enable --now	Enable at boot and start immediately
restart	Stop and start service
status	Show current state and recent status details
journalctl -u	Read logs for a specific service
disable	Remove boot-time enablement

Production habit — Use systemctl and journalctl for managed services. Avoid starting duplicate Node processes with `&`, `nohup` or `sleep` as a substitute for service management.

6. MySQL database

6.1 What is a database?

A database is an organised collection of data. MySQL is a relational database management system (RDBMS): data is stored in tables with rows and columns, and SQL is used to query and change it.

Real example: A bank keeps customer, account and transaction records in related tables. Our expense app keeps users, categories and expenses in tables.

6.2 Database vs DBMS vs schema vs table

Word	Meaning	Real-world analogy

Database	A named collection of related data objects	A filing cabinet for the Expense application
DBMS	Software that manages databases	The filing-room system and staff managing records
Schema	Structure/namespace of database objects	The filing plan and folder layout
Table	Rows and columns holding a type of data	One ledger/register
Row/record	One item in a table	One expense entry
Column/field	One attribute of a row	Date, amount or category
Primary key	Unique identifier for a row	Unique receipt/reference number
Foreign key	Reference to a row in another table	Expense's user_id points to a user
Index	Data structure that speeds up selected lookups	An index in a book to find a topic faster
SQL	Language used to query/manage relational data	Instructions to search or update the ledger

6.3 Install MySQL (verify OS/package first)

```
cat /etc/os-release
sudo dnf search mysql-server
# Install the package supported by your OS/repository:
sudo dnf install -y mysql-server
sudo systemctl enable --now mysqld
sudo systemctl status mysqld
```

Package names and repositories differ between Amazon Linux versions, RHEL and other distributions. Follow the supported installation procedure for the exact OS and MySQL version.

6.4 Secure MySQL

```
sudo mysql_secure_installation
sudo mysql
```

Use the installed version's security procedure. Remove anonymous accounts, restrict remote root login, remove test objects where appropriate and use a strong administrator password.

6.5 Create database and application user

```
CREATE DATABASE transactions
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

CREATE USER 'expense_app'@'<BACKEND_PRIVATE_IP>'
  IDENTIFIED BY '<LONG_RANDOM_SECRET>';

GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, ALTER, INDEX
  ON transactions.* TO 'expense_app'@'<BACKEND_PRIVATE_IP>';

FLUSH PRIVILEGES;
```

This is an example least-privilege grant. The actual privileges should match schema migration and runtime needs. If schema changes are done separately, the runtime account may need fewer privileges. Use a controlled host/subnet pattern appropriate to your environment.

6.6 Load schema

A schema file defines database objects such as tables, columns, indexes and constraints. Inspect it before running it.

```
# Example only: verify this file exists and is trusted
mysql -h <DB_PRIVATE_IP> -u <DB_ADMIN_USER> -p transactions < /app/schema/backend.sql
```

6.7 Database connectivity

```
# On DB host: check listener
sudo ss -lntp | grep 3306


# From backend host: test TCP
nc -vz <DB_PRIVATE_IP> 3306


# Test database login
mysql -h <DB_PRIVATE_IP> -u expense_app -p transactions
```

If TCP fails, investigate routes, security groups, network ACLs, host firewall, MySQL listen/bind configuration and DNS. If TCP works but login fails, investigate username, password, allowed host and grants.

6.8 Why private IP/private DNS?

The database is an internal dependency. It should not need a public internet address. A private DNS name such as mysql.internal.example.com can map to the database private IP, so the application does not need a hard-coded address.

Never — Do not publish a MySQL public DNS record or open port 3306 to the internet for this project.

7. Frontend and Nginx

7.1 What is a frontend?

The frontend is the part users see and interact with: pages, buttons, forms, tables, CSS and browser-side JavaScript. It commonly runs in the user's browser after static files are downloaded.

7.2 What is a web server?

A web server accepts HTTP requests and returns web content or forwards requests to another service. Nginx and Apache HTTP Server are examples.

7.3 What is Nginx?

Nginx is web server and reverse-proxy software. In this project it can serve frontend files and forward API requests to the backend.

7.4 Install Nginx

```
sudo dnf install -y nginx
sudo systemctl enable --now nginx
sudo systemctl status nginx
```

7.5 Deploy static frontend

```
# Example only: use your actual trusted build/release artifact
sudo mkdir -p /usr/share/nginx/html
# Copy/extract the frontend build files into this directory.
sudo chown -R root:root /usr/share/nginx/html
sudo find /usr/share/nginx/html -type d -exec chmod 755 {} \;
sudo find /usr/share/nginx/html -type f -exec chmod 644 {} \;
```

For a React/Vue/Angular-style application, the deployment artifact is often a built static directory. The source code and build instructions determine the correct output directory.

7.6 Nginx configuration: serve frontend and proxy API

```
# /etc/nginx/conf.d/expense.conf
server {
    listen 80;
    server_name <DOMAIN> www.<DOMAIN>;

    root /usr/share/nginx/html;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }

    location /api/ {
        proxy_pass http://<BACKEND_PRIVATE_IP>:8080;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

With this example, the proxy_pass has no trailing slash, so the original /api/ path is generally preserved. If proxy_pass ends with a slash, Nginx URI replacement behaviour differs. Match the setting to the routes implemented by the backend.

7.7 Validate Nginx

```
sudo nginx -t
sudo systemctl reload nginx
sudo systemctl status nginx
curl -I http://127.0.0.1/
```

nginx -t checks configuration syntax and referenced files. Reload applies valid configuration without a full stop/start.

7.8 Static content vs dynamic content

Static content is served as files, such as HTML, CSS, images and built JavaScript. Dynamic content is generated or retrieved when a request is processed, commonly by a backend API.

Real example: The login page layout is static frontend content. The user's current expense list is dynamic data retrieved from the backend.

8. Proxy concepts: forward proxy, reverse proxy and load balancer

8.1 What is a proxy?

A proxy is an intermediary that receives a request and makes or forwards a request on behalf of another party. The exact role depends on which side it represents.

8.2 Forward proxy

A forward proxy acts on behalf of clients. Client devices send outbound requests through it. It may enforce company internet policy, filter websites, log access or cache content.

Real example: An employee laptop is configured to use the company's secure web gateway. When the employee visits an external website, the gateway makes the outbound request on the employee's behalf.

```
Employee laptop -> Corporate forward proxy -> Internet website
```

8.3 Reverse proxy

A reverse proxy acts on behalf of backend servers. Clients connect to the proxy; the proxy forwards requests to internal application servers. It can hide backend addresses, terminate TLS, route paths, compress/cache responses and balance traffic.

Real example: A customer visits shop.example.com. Nginx accepts the request and forwards /api/orders to an internal Node.js server. The customer does not need to know the backend private IP.

```
Browser -> Nginx reverse proxy -> Node.js backend
```

8.4 Forward proxy vs reverse proxy

Question	Forward proxy	Reverse proxy
Represents	Client/user side	Server/application side
Typical location	Client egress path	In front of application servers
Who configures/uses it?	Often client or organisation configures it	Application owner configures it
Example	Corporate web filtering proxy	Nginx or ALB in front of backend
Main purpose	Control outbound client traffic	Protect, route and manage inbound server traffic

8.5 What is load balancing?

Load balancing distributes incoming requests across multiple servers (targets) to improve capacity and availability. A load balancer can stop sending new requests to targets that fail health checks.

Real example: A shopping site receives thousands of visitors. Instead of sending every request to one web server, a load balancer distributes requests across several servers.

8.6 Load balancing algorithms

Algorithm	Meaning	Example
Round robin	Rotate through targets in order	Request 1 to server A, 2 to B, 3 to A
Least connections	Prefer target with fewer active connections	Send next request to the less-busy server
Weighted	Give some targets a larger share	Large server receives more traffic
IP hash/sticky behaviour	Use a key to keep requests associated with a target	A client may keep returning to the same server

AWS ALB uses its own routing and load-distribution behaviour; do not assume it uses a simple round-robin algorithm in every case.

8.7 Why two frontend servers?

Two frontend instances reduce dependence on one machine and allow traffic to continue if one target becomes unhealthy, provided the ALB and remaining target are healthy and have enough capacity.

Important — Two EC2 instances alone do not create high availability. They need appropriate subnets/AZs, health checks, correct security groups, matching application versions and a load balancer.

9. DNS explained from the beginning

9.1 What is DNS?

DNS stands for Domain Name System. It is a distributed naming system that lets users access services using names instead of memorising IP addresses.

Real example: People remember a business name and phone contact more easily than a long number. DNS is like a distributed internet directory that helps find the network destination for a domain.

9.2 Why do we need DNS?

Server IPs can change because instances are replaced, scaled or moved. A stable domain name lets users continue using the same name while DNS records are updated to point to the current service endpoint.

9.3 Domain name parts

```
www.example.com.
|   |       |  |
|   |       |  +-- root (implicit final dot)
|   |       +----- TLD: com
|   +-------------- registered domain: example
+------------------ subdomain/host label: www
```

A domain is a hierarchical name. example.com is a registered domain under .com. www.example.com is a hostname/subdomain beneath it.

9.4 Who manages domain names?

Organisation/role	Responsibility	Example
ICANN	Coordinates policies and functions for global unique identifiers, including domain-name system coordination	Global coordination role
Registry operator	Maintains authoritative registration data for a TLD	Operator of a TLD such as .com
Registrar	Retail provider through which a customer registers a domain	A domain registration company
Registrant	Person or organisation registering the domain	Your company/project owner
DNS hosting provider	Hosts authoritative DNS records for the domain	Amazon Route 53, Cloudflare or registrar DNS

A registrar and DNS host are not necessarily the same company. Buying a domain does not automatically mean its DNS is hosted at the same provider.

9.5 DNS resolution: every step

When a user types https://www.example.com, the browser needs to find where that name leads. A simplified lookup proceeds as follows:

16. The browser and operating system check relevant local caches and configuration.
17. The device sends a DNS question to its configured recursive resolver, often supplied by the ISP, enterprise network or a public DNS provider.
18. If the recursive resolver has a valid cached answer, it returns it immediately.
19. If not cached, the resolver asks a DNS root server where to find the relevant TLD name servers.
20. The resolver asks a TLD server (for example, .com) which authoritative name servers serve example.com.

21. The resolver asks one of the domain's authoritative name servers for the requested record, such as www.example.com A/AAAA or an alias.
22. The authoritative server returns the record or alias information. The resolver caches the result according to TTL and returns the answer to the client.
23. The client then connects to the returned destination. DNS lookup is complete; the browser still needs to establish the network and HTTPS connection.

Important distinction — Root and TLD servers usually do not return your website's EC2 address. They direct the resolver to the next authoritative level. The domain's authoritative DNS server returns the actual record.

9.6 Recursive resolver vs authoritative DNS

DNS role	What it does	Analogy
Stub resolver	Small client-side component that asks a configured resolver	You ask a directory service
Recursive resolver	Finds the answer by checking cache and querying DNS hierarchy	A researcher who contacts other directories for you
Root server	Points toward TLD name servers	Points to the correct top-level directory
TLD server	Points toward authoritative servers for a registered domain	Points to the directory for that organisation/name
Authoritative server	Publishes the actual DNS records for a zone	The official record keeper

9.7 DNS record types

Record	Purpose	Real-world example
A	Maps hostname to IPv4 address	app.example.com -> 203.0.113.10
AAAA	Maps hostname to IPv6 address	app.example.com -> IPv6 address
CNAME	Aliases one hostname to another hostname	www.example.com -> app.example.com
MX	Identifies mail servers for a domain	Mail provider receives @example.com email
NS	Identifies authoritative name servers for a zone	Delegation to Route 53 name servers
TXT	Text data used for verification and email security policies	Domain ownership, SPF, DKIM, DMARC
SOA	Zone authority and timing/serial metadata	Primary metadata for the DNS zone
PTR	Reverse DNS mapping from address to name	IP address lookup to hostname
SRV	Identifies host/port for certain services	Service discovery for supported protocols

9.8 What is TTL?

TTL means Time To Live. In DNS it tells caching resolvers how long a record may be reused before they should query again.

Example: A record has TTL 300 seconds. A resolver may cache it for five minutes. If you change the record, a resolver that already cached the old answer may continue using it until its cached TTL expires.

Short TTL can make planned DNS changes take effect sooner after caches expire, but increases query volume. Long TTL reduces repeated queries but can delay changes. TTL does not force every device worldwide to update at exactly the same second.

9.9 DNS caching

DNS answers may be cached by browsers, operating systems, local network resolvers and recursive DNS providers. Different caches can expire at different times. This is why a DNS change may appear at different times for different users.

9.10 DNS does not do these things

- DNS does not install Nginx or start your backend.
- DNS does not open a security group port.
- DNS does not make a private IP reachable from the internet.
- DNS does not encrypt HTTP traffic; HTTPS/TLS does that.
- DNS does not itself balance application load unless a DNS-based routing policy is used; an ALB performs application traffic distribution.

9.11 Useful DNS commands

```
# Query IPv4 record
dig <DOMAIN> A

# Query IPv6
dig <DOMAIN> AAAA

# Find authoritative name servers
dig <DOMAIN> NS

# Trace delegation from root downward
dig +trace <DOMAIN>

# Query a specific resolver
dig @1.1.1.1 <DOMAIN> A

# Basic host lookup
nslookup <DOMAIN>

# Resolve a private service name from the backend host
getent hosts mysql.internal.example.com
```

10. Domain registration and Route 53

10.1 What is Amazon Route 53?

Amazon Route 53 is AWS's scalable DNS and domain registration service. It can host public or private DNS zones, answer DNS queries and provide routing policies. Domain registration is a separate capability from hosting DNS records.

10.2 What is a hosted zone?

A hosted zone is a container for DNS records for a domain. A public hosted zone answers public DNS queries. A private hosted zone answers queries from associated VPCs.

10.3 Public hosted zone vs private hosted zone

Public hosted zone	Private hosted zone
Used for names resolvable on the public internet.	Used for internal names inside associated VPCs.
Example: www.example.com points to public ALB.	Example: mysql.internal.example.com points to DB private IP.
Do not publish private infrastructure details unnecessarily.	Not directly resolvable by arbitrary internet users.

10.4 Create public DNS hosting

24. Open Route 53 in the correct AWS account and Region-independent global service console.
25. Choose Hosted zones and create a public hosted zone for the exact registered domain.
26. Record the four Route 53 name-server values created for the zone.
27. At the domain registrar, replace the current authoritative name-server delegation with the Route 53 name servers.
28. Wait for delegation and caches to update. Verify using dig NS <DOMAIN> and dig +trace <DOMAIN>.
29. Create the website record after the ALB exists. Prefer an Alias A/AAAA record to the ALB where supported.

Common misunderstanding — Creating a Route 53 hosted zone does not transfer the domain registration to AWS. Updating name-server delegation and transferring registration are different actions.

10.5 What is an Alias record?

A Route 53 Alias record is an AWS-specific DNS feature that can point a name, including a zone apex in supported cases, to selected AWS resources such as an ALB. It behaves like an address answer to DNS clients while Route 53 manages the target relationship.

10.6 Website record design

Name	Record	Target
example.com (root)	A – Alias	Application Load Balancer
www.example.com	A – Alias	Same Application Load Balancer, or supported alias target
mysql.internal.example.com	A record in private zone	DB private IP

10.7 Private hosted zone for database

Create a private hosted zone such as internal.example.com and associate it with the VPC where the backend runs. Add mysql.internal.example.com pointing to the DB private IP. Set DB_HOST in the backend configuration to this name.

10.8 DNS troubleshooting

Symptom	What to check
NXDOMAIN	Does the record exist? Is the domain delegated to the intended authoritative DNS?
Old IP still returned	TTL/caches, resolver, record at authoritative DNS
dig works but website fails	DNS is only one step; check ALB, listener, security groups, target health and app
Private name does not resolve	Private zone VPC association, VPC DNS support/hostnames and query source
Name servers differ from Route 53 zone	Registrar delegation may not have been updated

11. AWS Application Load Balancer (ALB)

11.1 What is a load balancer?

A load balancer is a service that accepts traffic and distributes it across registered targets. AWS Elastic Load Balancing provides managed load-balancing services.

11.2 What is an Application Load Balancer?

An Application Load Balancer is a Layer 7 load balancer for HTTP and HTTPS. It can route based on hostnames, URL paths, headers and other supported conditions.

Real example: /api/* can be routed to an API target group while / can be routed to a frontend target group. In this specific project design, the ALB can send all website traffic to the frontend tier, and Nginx can proxy API requests to the backend.

11.3 Layer 4 vs Layer 7

Layer 4 load balancer	Layer 7 load balancer
Works at transport layer using IP/port and TCP/UDP details.	Understands application protocols such as HTTP/HTTPS.
Does not normally inspect URL path to make routing decisions.	Can route based on host, path and HTTP rules.
AWS Network Load Balancer is a common example.	AWS Application Load Balancer is a common example.

11.4 ALB terminology

Term	Meaning	Real example
Listener	Process that checks for connection requests on a protocol/port	HTTPS listener on port 443
Listener rule	Condition/action that determines where a request goes	If host is app.example.com, forward to frontend target group
Default action	Action used if no higher-priority rule matches	Forward to default target group
Target group	Collection of registered targets and health-check settings	Frontend EC2 instances on port 80
Target	Destination registered in target group	Frontend EC2 instance
Health check	Periodic request/test to see if target is healthy	GET /health.html
Availability Zone	Separate AWS infrastructure location within a Region	Frontend instances distributed across two AZs
TLS certificate	Certificate used to secure HTTPS connections	ACM certificate for example.com

11.5 ALB request flow

30. Browser resolves the domain through DNS to the ALB endpoint.
31. Browser connects to ALB on TCP 443 and negotiates TLS.
32. The ALB HTTPS listener receives the request.
33. Listener rules evaluate host/path conditions.
34. The selected target group provides eligible healthy targets.
35. The ALB forwards the request to one healthy frontend EC2 target on its configured port.
36. The frontend responds, and the ALB returns the response to the browser.

11.6 Target group health check

A health check is a periodic test that helps the load balancer decide whether a target can receive traffic. It can use a path such as / or /health.html, a protocol and a port. The path must return a configured success status.

If the health check tests /health.html but the frontend does not have that file, the target can become unhealthy even though Nginx is running.

11.7 Recommended lab ALB setup

37. Create an internet-facing ALB in at least two public subnets in different Availability Zones.
38. Create a target group for frontend instances, protocol HTTP, port 80, target type Instances (for this EC2 lab).
39. Set a health-check path that exists and returns a success code.
40. Register frontend EC2 instance 1 and instance 2.
41. Create an ALB security group that allows HTTPS 443 from intended users.
42. Create a frontend security group that allows HTTP 80 only from the ALB security group.
43. Create an HTTPS listener using an ACM certificate for the domain.
44. Optionally create an HTTP 80 listener that redirects to HTTPS 443.
45. Create a Route 53 Alias record to the ALB.

11.8 ALB vs Nginx reverse proxy

ALB	Nginx
AWS-managed service; AWS manages underlying load-balancer infrastructure.	Software installed and maintained on your EC2/server.
Provides target groups, health checks, listeners, rules and AWS integrations.	Provides flexible web serving, reverse proxy, caching and custom routing.
Good public entry point for AWS workloads.	Good frontend web server and internal reverse proxy.
Can use ACM integration for TLS listener certificates.	Can terminate TLS when certificates and config are managed on the host.

Recommended design — Use ALB as the public internet entry point and Nginx on frontend instances to serve files and proxy API requests to the backend. This keeps the architecture clear and avoids exposing backend or database ports publicly.

12. Complete practical deployment runbook

The following sequence is a controlled lab plan. Record the actual values in a private inventory file, not in a public repository.

12.1 Architecture and value sheet

Placeholder	Example meaning	Your value
<REGION>	AWS Region, e.g. ap-south-1	
<VPC_ID>	Project VPC	
<DOMAIN>	Registered domain	
<ALB_DNS>	DNS name generated by ALB	
<FRONTEND_1_PRIVATE_IP>	Frontend 1 private IP	
<FRONTEND_2_PRIVATE_IP>	Frontend 2 private IP	
<BACKEND_PRIVATE_IP>	Backend private IP	
<DB_PRIVATE_IP>	MySQL private IP	
<BACKEND_PORT>	Node.js listen port, e.g. 8080	
<DB_NAME>	Application database name	transactions

12.2 Phase A – Prepare the AWS network

46. Select a VPC and at least two Availability Zones for frontend/ALB availability.
47. Create or identify public subnets for the internet-facing ALB.
48. Place backend and database instances in private subnets where possible.
49. Check route tables: public subnets need a route to an internet gateway; private subnets need planned outbound access if package installation is required.
50. Create role-specific security groups using the least-privilege matrix in section 2.9.
51. Plan access to private instances through Systems Manager Session Manager or a tightly controlled bastion/admin path.

12.3 Phase B – Build MySQL database host

52. Launch a database EC2 instance in a private subnet and attach DB-SG.
53. Connect using an approved administration path.
54. Install a supported MySQL version and start/enable the service.
55. Secure MySQL and confirm it listens on the intended private interface.
56. Create the transactions database and least-privilege application user.
57. Import the application schema after inspecting it.
58. From backend network, test TCP 3306 and database authentication.
59. Create private DNS for the DB if using Route 53 private hosted zone.

12.4 Phase C – Build backend EC2

60. Launch backend EC2 in a private subnet and attach Backend-SG.
61. Install the Node.js version required by the application.
62. Create the service account and application directory.
63. Copy or download the trusted backend release and inspect its files.
64. Run npm ci --omit=dev when a consistent package-lock.json is present.
65. Configure environment variables with DB private endpoint and secret values.
66. Test the application in foreground and verify local health/API route.
67. Create backend.service and start it through systemd.
68. Check systemctl status, journalctl and listening port.
69. From the frontend host, test the backend port and API route.

12.5 Phase D – Build frontend EC2 instance 1

70. Launch frontend EC2 and attach Frontend-SG.
71. Install Nginx and enable/start it.
72. Deploy the frontend build into the correct document root.

73. Configure Nginx to serve the frontend and proxy API paths to backend private address.
74. Run nginx -t and reload Nginx.
75. Test the home page locally and from another allowed host.
76. Test the API through Nginx and check backend logs.

12.6 Phase E – Build frontend EC2 instance 2

77. Launch a second frontend EC2 in another Availability Zone if available.
78. Install the same Nginx version/configuration and deploy the same frontend release.
79. Ensure both instances use the correct backend endpoint.
80. Test the same URL and health-check path on both instances.
81. Confirm both instances have matching configuration and release version.

12.7 Phase F – Create ALB

82. Create the frontend target group on HTTP port 80.
83. Set health-check protocol, path and success matcher.
84. Register both frontend EC2 instances.
85. Create an internet-facing ALB across at least two public subnets.
86. Attach ALB-SG and create HTTPS 443 listener with ACM certificate.
87. Add HTTP 80 redirect to HTTPS if desired.
88. Forward the HTTPS listener to the frontend target group.
89. Wait for both targets to become healthy and investigate failures.

12.8 Phase G – Connect domain

90. Create or confirm the Route 53 public hosted zone.
91. Confirm registrar name-server delegation points to the authoritative Route 53 name servers.
92. Create Route 53 Alias A/AAAA record to the ALB as appropriate.
93. Check DNS using dig and test HTTPS from a client network.
94. Confirm certificate covers the exact domain names users visit.

12.9 Phase H – End-to-end test

95. Open https://<DOMAIN> in a browser.
96. Confirm the frontend loads and browser developer tools show no critical errors.
97. Sign in with a test account.
98. Create an expense using valid test data.
99. Confirm API response and database row.
100. Read the expense list and verify the new record appears.
101. Update and delete a test record if those operations are supported.
102. Stop one frontend target in the lab and verify the ALB routes to the remaining healthy target.
103. Restore the target and verify it returns to healthy status.

13. Testing and troubleshooting

13.1 Test each layer separately

Layer	Test	What success tells you
DNS	dig <DOMAIN> A / Alias lookup	Name resolves to intended public entry point
ALB	Check listener and target health	ALB accepts traffic and has healthy target(s)
Frontend	curl -I http://127.0.0.1/	Nginx returns frontend response locally
Frontend → backend	curl http://<BACKEND_PRIVATE_IP>:8080/<REAL_ROUTE>	Network path and backend route work
Backend → DB	nc -vz <DB_PRIVATE_IP> 3306 then mysql login	TCP and DB authentication work
API	Postman/curl	Request, business logic and response work

13.2 Common symptoms

Symptom	Likely area	What to inspect
DNS name does not resolve	DNS/registrar	Authoritative NS, record, hosted zone, TTL
ALB target unhealthy	Health check/network/web server	Path, port, Nginx, frontend SG from ALB SG
Website timeout	ALB/network	Listener, subnet routes, ALB SG, target health
502 Bad Gateway	Reverse proxy/upstream	Nginx proxy_pass, backend service, port and route
504 Gateway Timeout	Slow/unreachable upstream	Backend logs, DB latency, network, timeout
Frontend works but API fails	API/proxy/CORS	Browser console, API base URL, Nginx location, backend route
Backend cannot connect to DB	DB/network/credentials	DB listener, SG 3306, private DNS, grants, password
401 Unauthorized	Authentication	Token/session, expiry, auth middleware
403 Forbidden	Authorization	User permissions and resource ownership
404 Not Found	URL/routing	Correct API path, Nginx URI mapping, backend route
500 Internal Server Error	Application	Backend logs, stack trace, input and DB error

13.3 Useful Linux commands

```
# OS and resources
cat /etc/os-release
hostnamectl
df -h
free -h
uptime

# Network
ip -br addr
ip route
sudo ss -lntup
nc -vz <HOST> <PORT>

# Processes
ps -ef | grep node
pgrep -a node

# Services and logs
sudo systemctl status backend
sudo systemctl status nginx
sudo journalctl -u backend -n 100 --no-pager
sudo journalctl -u nginx -n 100 --no-pager

# Nginx
sudo nginx -t
sudo nginx -T
sudo tail -f /var/log/nginx/access.log
sudo tail -f /var/log/nginx/error.log
```

```
# HTTP and DNS
curl -i http://127.0.0.1:<PORT>/<REAL_PATH>
curl -I https://<DOMAIN>
dig <DOMAIN> A
dig <DOMAIN> NS
getent hosts <PRIVATE_DNS_NAME>
```

13.4 Troubleshooting method

104. Define the symptom precisely: which URL, source, time and error code?
105. Identify the failing hop in the architecture: DNS, ALB, frontend, backend or database.
106. Check whether the process is running and listening on the expected port.
107. Check whether the source host can reach the destination port.
108. Check application logs at the failing layer.
109. Change one thing at a time and repeat the same test.
110. Record root cause, fix, validation and rollback plan.

14. Security and production readiness

Area	Recommended practice	Reason
Secrets	Use Secrets Manager/Parameter Store or protected files; rotate exposed values	Prevent credential leakage
IAM	Attach least-privilege instance roles; avoid long-lived access keys on EC2	Reduce credential risk
Network	Private backend and DB; security-group references; no public MySQL	Limit attack surface
TLS	Use HTTPS with ACM certificate on ALB	Encrypt browser traffic
Linux	Run app as non-root; patch OS; restrict SSH	Reduce host compromise impact
Application	Validate input, authenticate and authorize every protected operation	Prevent abuse and data exposure
Database	Least-privilege DB user, backups, tested restore	Protect data and recovery
Availability	Multiple AZs, target health checks, capacity planning	Reduce single-instance failure
Monitoring	CloudWatch metrics/logs and alarms	Detect failures and capacity issues
Deployment	Versioned releases, rollback plan and tested config	Reduce deployment risk

Credential hygiene — If a real database password, admin token or API secret was exposed in a notebook photo, shell history or chat, rotate it. Do not reuse it in the next deployment.

15. Large technical glossary with real-world examples

Term	Meaning	Real-world example
Architecture	The structure of components and how they interact.	A building plan showing rooms, doors, wiring and water pipes; in software it shows browser, ALB, frontend, backend and DB.
Client	A program/device that requests a service.	Chrome requesting a page from a website.
Server	A computer/process that provides a service.	Node.js responding to an expense API request.
Client-server architecture	A model where clients request and servers respond.	A mobile banking app requests account details from bank servers.
Frontend	User-facing interface.	Expense form, buttons, dashboard and tables.
Backend	Server-side business logic and API.	Checks user identity and saves expenses.
Database	Organised persistent data store.	Ledger containing expense records.
DBMS	Software that manages databases.	MySQL manages tables, users and queries.
API	A defined interface for software components to	Food app asks a restaurant service for menu

	communicate.	items.
API endpoint	Specific API path/method that performs an operation.	GET /api/expenses retrieves expense records.
REST	Style for designing resource-oriented APIs.	/api/expenses with GET, POST, PATCH and DELETE.
HTTP	Protocol for web request/response communication.	Browser sends GET request to fetch a page.
HTTPS	HTTP protected by TLS encryption/authentication.	Bank website URL begins with https://.
Request	Message sent by a client to a server.	POST request containing an expense amount.
Response	Message returned by a server.	201 Created with new expense ID.
Header	Metadata attached to HTTP request/response.	Content-Type: application/json.
Body	Main payload of an HTTP message.	JSON data containing description and amount.
JSON	Text format for structured data exchange.	{"amount":1200,"category":"Travel"}.
CRUD	Create, Read, Update, Delete.	Add, view, edit and remove an expense.
Authentication	Verifying who a user is.	Checking a login password or token.
Authorization	Checking what an authenticated user may do.	User can view only their own expenses.
Middleware	Code that runs in a request pipeline before/after route logic.	Authentication middleware checks a token before the expense route.
Node.js	JavaScript runtime outside the browser.	Runs the backend index.js process.
Express	Node.js web framework.	Defines GET /api/expenses route.
npm	Node.js package-management command-line tool.	Installs Express and database driver.
Package	Reusable software library/module.	Express is a package.
Dependency	Package required by an application.	mysql2 is a dependency if backend uses it.
Runtime	Environment that executes a program.	Node.js runtime executes JavaScript.
Environment variable	External process configuration value.	DB_HOST tells backend which MySQL server to contact.
Secret	Sensitive credential or token.	Database password or JWT signing key.
Process	Running instance of a program.	Node.js backend process with a PID.
PID	Process ID assigned by Linux.	Used to identify a running Node process.
Daemon	Background process that provides a service.	A service process running without an interactive terminal.
systemd	Linux system/service manager.	Starts backend at boot and restarts it on failure.
Unit file	systemd configuration describing a service or other unit.	backend.service specifies user, command and restart policy.
Journal	systemd logging system.	journalctl displays backend service logs.
Nginx	Web server and reverse-proxy software.	Serves frontend files and forwards /api requests.
Web server	Software that handles HTTP requests and serves content.	Nginx serves index.html.
Static content	Files served without per-request business logic.	CSS, images and built JavaScript.
Dynamic content	Content generated or retrieved at request time.	Current user's expense list from MySQL.
Proxy	Intermediary that forwards requests.	A middle service between a browser and backend.
Forward proxy	Proxy representing clients for outbound requests.	Company web gateway filters employee internet access.
Reverse proxy	Proxy representing servers for inbound requests.	Nginx accepts browser traffic and forwards to Node.js.
Load balancer	Distributes traffic across multiple targets.	Website traffic spread across two frontend EC2 instances.
Reverse proxy vs load balancer	Reverse proxy forwards on behalf of servers; load balancer distributes across targets. One product can do both.	Nginx can reverse proxy and balance; ALB distributes across registered targets.
DNS	Distributed system that resolves names to records.	Finds where example.com points.
Domain	Hierarchical human-readable name.	example.com.
Subdomain	Name beneath a registered domain.	api.example.com.
TLD	Top-level domain at end of a domain.	.com, .in, .org.
Registrar	Company through which a domain is registered.	Provider where company renews example.com.
Registry	Operator maintaining TLD registration data.	Operator of the .com registry.
Registrant	Person or organisation that registers a domain.	Business that owns the domain registration.
Name server	DNS server that provides or points to DNS authority.	Route 53 authoritative name servers for a zone.

Authoritative DNS	Source that publishes the official records for a zone.	Route 53 answers A/AAAA/TXT records for example.com.
Recursive resolver	Resolver that finds answers by cache and querying DNS hierarchy.	ISP/public resolver used by a laptop.
Root DNS server	Top level of DNS delegation; points to TLD servers.	Directs resolver toward .com name servers.
TLD DNS server	Provides delegation to domain authoritative name servers.	Points resolver to example.com's authoritative servers.
DNS record	A data entry in a DNS zone.	A record for app.example.com.
A record	Maps a name to IPv4.	app.example.com -> 203.0.113.10.
AAAA record	Maps a name to IPv6.	app.example.com -> IPv6 address.
CNAME	Alias from one DNS name to another DNS name.	www.example.com points to app.example.com.
MX	Mail exchange record.	Directs email for example.com to mail provider.
TXT	Text record often used for verification and email security.	SPF or domain verification string.
NS	Name server record.	Lists authoritative DNS servers.
SOA	Start of Authority record with zone metadata.	Contains zone serial and timing values.
TTL	How long a DNS answer may be cached.	300-second TTL means cache may reuse answer for five minutes.
DNS propagation	Informal term for caches/delegation becoming updated across resolvers.	Some users still see old answer until cached TTL expires.
Hosted zone	Route 53 container for records belonging to a domain.	Public hosted zone for example.com.
Public hosted zone	DNS zone answerable publicly.	Public website record points to ALB.
Private hosted zone	DNS zone resolvable from associated VPCs.	mysql.internal.example.com resolves inside VPC.
Route 53 Alias	AWS record feature pointing a name to supported AWS resource.	Root domain Alias to ALB.
IP address	Network address used to route packets.	Backend private IP 10.0.2.15.
Private IP	Address used within private network routing.	DB address accessible from backend VPC.
Public IP	Internet-routable address, subject to controls.	ALB public endpoint address.
Port	Number identifying a transport service endpoint.	443 for HTTPS, 3306 for MySQL.
Protocol	Rules for communication.	TCP, UDP, HTTP, DNS and TLS.
TCP	Reliable, ordered transport protocol.	HTTPS connection commonly uses TCP.
UDP	Datagram transport protocol without TCP-style reliability guarantees.	DNS queries often use UDP.
TLS	Cryptographic protocol securing a connection.	Encrypts browser-to-ALB HTTPS traffic.
Certificate	Digital identity credential used in TLS.	ACM certificate for example.com.
Firewall	Traffic filtering control.	Allows only approved ports and sources.
Security Group	Stateful virtual firewall for AWS resources.	DB-SG allows 3306 from Backend-SG only.
Subnet	IP range within a VPC.	Public subnet for ALB; private subnet for DB.
VPC	Logically isolated AWS network.	Project network containing ALB, EC2 and database.
Route table	Rules that determine where subnet traffic is routed.	Public subnet default route to internet gateway.
Internet Gateway	VPC component enabling internet connectivity for public resources when routing and public addressing are configured.	ALB in public subnets reaches/receives internet traffic.
NAT Gateway	Managed service allowing private subnet resources to initiate internet connections without accepting unsolicited inbound internet connections.	Private EC2 downloads OS packages.
Availability Zone	Isolated location within an AWS Region.	Frontend instances spread across two AZs.
Region	Geographic AWS area containing multiple AZs.	ap-south-1 Mumbai.
EC2	AWS virtual server service.	Linux VM running backend.
AMI	Amazon Machine Image used to launch EC2 instances.	Reusable server image with OS and software.
Instance type	EC2 compute/memory/network capacity selection.	t3.micro is an example instance type.
IAM	AWS identity and access management service.	Controls which AWS actions an EC2 role may perform.
IAM role	Identity with permissions assumed by a service or user.	EC2 role reads a secret from Secrets Manager.

Least privilege	Give only permissions needed for the task.	Backend DB user can read/write application tables but not administer MySQL.
ALB	Application Load Balancer, Layer 7 HTTP/HTTPS load balancer.	Public HTTPS entry point for the website.
Listener	ALB process listening on a protocol/port.	HTTPS listener on 443.
Listener rule	Condition/action used to route requests.	Path /api/* forwards to API target group.
Target group	Set of targets and health-check settings.	Two frontend EC2 instances on port 80.
Target	Registered destination receiving traffic.	Frontend EC2 instance.
Health check	Periodic test of target availability.	ALB requests /health.html and expects success.
TLS termination	Decrypting TLS at a proxy/load balancer.	ALB handles HTTPS and forwards HTTP inside a protected VPC path.
Availability	Ability of a service to remain accessible.	Second frontend continues serving if first fails.
Scalability	Ability to handle increased load by adding capacity.	Add frontend instances to target group.
Horizontal scaling	Add more instances.	Increase frontend count from two to four.
Vertical scaling	Increase capacity of one machine.	Move to larger EC2 instance type.
Monitoring	Collecting metrics/logs/traces and checking health.	CloudWatch alarms on CPU and ALB 5xx.
Logging	Recording events for investigation.	journalctl backend logs.
Metric	Numerical measurement over time.	CPU percentage, request count, latency.
Latency	Time taken for an operation/request.	API response takes 250 ms.
Throughput	Amount of work processed per unit time.	Requests per second.
Timeout	Maximum time allowed before a request is abandoned.	ALB waits for backend only up to configured timeout.
502 Bad Gateway	Proxy got invalid response from upstream.	Nginx cannot get valid response from Node.js.
504 Gateway Timeout	Proxy did not receive upstream response in time.	Backend/database too slow or unreachable.
Deployment	Installing/releasing application version into an environment.	Copy new frontend build and restart backend.
Rollback	Return to a previous known-good release/configuration.	Restore previous app version after failed deployment.
CI/CD	Continuous Integration and Continuous Delivery/Deployment practices.	Pipeline tests and deploys a code change.
Artifact	Packaged output of a build.	Frontend dist folder or backend release archive.
Health endpoint	API path used to report service health.	/health returns status if implemented.
CORS	Browser security mechanism controlling cross-origin requests.	Frontend domain calling API on another domain may need CORS headers.
SQL injection	Attack where untrusted input changes SQL query structure.	Use parameterized queries rather than concatenating user input.
Backup	Copy used to recover data.	Automated MySQL backup stored separately.
Restore	Recovering data from backup.	Rebuild database after accidental deletion.

16. Practical exercises

Exercise 1 – Client and server

Use curl to request the backend health route. Identify which machine is client and which process is server.

Exercise 2 – Ports

Run ss -lntup on each host. Identify Nginx, Node.js and MySQL listening ports.

Exercise 3 – API

Use Postman to create and retrieve a test expense. Record method, URL, headers, body and status code.

Exercise 4 – systemd

Restart backend with systemctl. Confirm it returns to active state and inspect journalctl.

Exercise 5 – Database

Connect from backend host using the application DB user. Run SHOW TABLES and a safe SELECT.

Exercise 6 – Reverse proxy

Call the API through the frontend Nginx URL and compare with calling backend directly.

Exercise 7 – DNS

Run dig +trace for your domain. Identify root, TLD and authoritative delegation.

Exercise 8 – TTL

Inspect the record TTL and explain why different resolvers may show old answers temporarily.

Exercise 9 – ALB health

Stop one frontend service in the lab and observe target health. Restart it and observe recovery.

Exercise 10 – Failure drill

Intentionally use a wrong backend port in a lab config, diagnose 502, correct it and validate.

17. Interview questions with short answers

Question	Answer
What is client-server architecture?	A model where a client sends requests and a server provides responses over a network.
What is an API?	A defined interface through which software components communicate.
What is REST?	An architectural style commonly using resource URLs and HTTP methods.
What is the difference between authentication and authorization?	Authentication verifies identity; authorization checks permissions.
What is a reverse proxy?	An intermediary in front of servers that accepts client requests and forwards them to backend services.
What is a forward proxy?	An intermediary that represents clients for outbound requests.
What is a load balancer?	A service that distributes requests across multiple targets.
What is DNS?	A distributed naming system that resolves domain names to DNS records.
What happens during DNS resolution?	The recursive resolver uses cache or queries root, TLD and authoritative DNS servers.
What is TTL?	The time a DNS answer can remain cached.
What is the difference between A and CNAME?	A maps a name to IPv4; CNAME aliases one DNS name to another.
What is a Route 53 hosted zone?	A container for DNS records for a domain.
What is an ALB listener?	A process that checks for connection requests on a configured protocol and port.
What is a target group?	A group of registered destinations and health-check settings.
Why are health checks needed?	To detect targets that should not receive new traffic.
Why keep MySQL private?	To prevent direct public access to sensitive data services.
Why use systemd?	To manage service startup, restart policy, state and logs.
What is npm ci?	A repeatable dependency installation command based on the lock file.
What is a 502?	A gateway/proxy received an invalid response from an upstream service.
What is a 504?	A gateway/proxy timed out waiting for an upstream response.

18. Final deployment checklist

- ☐ Backend starts successfully under systemd and survives a reboot.
- ☐ Backend runs as a non-root service account.
- ☐ Secrets are not committed to Git or stored in readable world-accessible files.
- ☐ Backend can resolve and connect to MySQL using private DNS/IP.

- ☐ MySQL is not publicly reachable and allows only the backend security group.
- ☐ Nginx serves the correct frontend build.
- ☐ Nginx API proxy path matches the backend routes.
- ☐ Both frontend EC2 instances have matching configuration and release version.
- ☐ ALB is deployed across at least two Availability Zones.
- ☐ ALB HTTPS listener uses a valid ACM certificate.
- ☐ Both frontend targets pass health checks.
- ☐ Route 53 public record points to the ALB.
- ☐ Registrar delegation points to the intended authoritative name servers.
- ☐ CRUD operations work through the public HTTPS domain.
- ☐ Logs are available and a basic monitoring/alerting plan exists.
- ☐ Database backup and restore process is understood and tested.
- ☐ Lab resources are deleted when practice is complete.

19. A final mental model to remember

Think of the complete project like a real office:

Technology	Office analogy
DNS	Directory that tells a visitor where the organisation's public entrance is
ALB	Reception desk that sends visitors to an available counter
Frontend/Nginx	Customer-facing counter and display board
API	Standard request form used to ask another department for an operation
Reverse proxy	Reception intermediary that forwards a request to the correct internal team
Backend	Staff who validate and process the request
MySQL	Secure ledger room where records are stored
Security Group	Access-control doors deciding which departments can enter
systemd	Facilities manager ensuring essential services start and restart
Logs/metrics	CCTV/event book and performance dashboard used to investigate issues

The most important troubleshooting habit is to follow the request hop by hop. Do not randomly restart every service. Prove DNS, then ALB, then frontend, then backend, then database. Find the first hop that fails; inspect its configuration, network access and logs.

End of guide — Build one layer at a time. Understand the purpose of each command before executing it. The goal is not merely to make the website work once; it is to understand why it works, how to verify it, and how to recover when one component fails.

20. DevOps Interview Preparation – Questions and Answers

Practical interview questions from Linux, networking, Git, CI/CD, Docker, AWS, Kubernetes, monitoring and troubleshooting.

Use these answers as a starting point. In an interview, explain the concept in your own words and connect it to the Expense Management deployment project. Interviewers often ask follow-up questions to check whether you have actually practised the technology.

A. DevOps fundamentals

Q. What is DevOps?

Answer: DevOps is a culture and set of practices that bring development and operations teams together. It focuses on automation, collaboration, continuous integration, continuous delivery, monitoring and faster, safer releases.

Q. Why is DevOps used?

Answer: It helps teams release software more frequently and reliably, reduce manual errors, shorten feedback cycles and improve collaboration between development, testing, security and operations.

Q. What is the difference between Agile and DevOps?

Answer: Agile is mainly a way to organise software development in iterative cycles. DevOps extends collaboration and automation across development, release, deployment and operations. They complement each other.

Q. What is CI?

Answer: Continuous Integration is the practice of frequently merging code changes into a shared repository and automatically building and testing them so integration problems are found early.

Q. What is CD?

Answer: Continuous Delivery keeps software in a deployable state and makes releases ready for production, often with a manual approval. Continuous Deployment automatically releases qualifying changes to production.

Q. What is Infrastructure as Code?

Answer: Infrastructure as Code manages infrastructure through version-controlled configuration files rather than manual console clicks. Terraform is a common example.

Q. What is configuration management?

Answer: It maintains consistent server configuration, packages, files and services. Ansible is commonly used to automate these tasks.

Q. What is a deployment pipeline?

Answer: A sequence of automated stages such as checkout, build, test, security scan, package, publish and deploy.

Q. What is a rollback?

Answer: A controlled return to a previously working application version or configuration when a new release causes problems.

Q. What is a blue-green deployment?

Answer: Two production-like environments are maintained. Traffic is switched from the old environment (blue) to the new one (green) after validation, allowing a quick switch back if needed.

B. Linux and shell scripting

Q. How do you check Linux OS version?

Answer: Use cat /etc/os-release. hostnamectl can also show operating-system and kernel information.

Q. How do you check disk usage?

Answer: Use df -h for filesystem usage and du -sh <directory> to check directory size.

Q. How do you check memory?

Answer: Use free -h. For live process-level inspection, use top or an approved monitoring tool.

Q. How do you check CPU and load?

Answer: Use top, uptime, mpstat or vmstat if installed. Remember that load average is not the same as CPU percentage.

Q. How do you find a process using a port?

Answer: Use sudo ss -lntp and inspect the process name/PID. lsof -i :<PORT> may also help if installed.

Q. How do you check a service?

Answer: Use systemctl status <service>. For logs, use journalctl -u <service> -n 100.

Q. What is the difference between soft link and hard link?

Answer: A symbolic link points to a path and can cross filesystems. A hard link is another directory entry for the same inode and normally cannot cross filesystem boundaries.

Q. What are file permissions?

Answer: Linux permissions define read, write and execute access for owner, group and others. chmod changes permissions and chown changes ownership.

Q. What is a cron job?

Answer: Cron schedules commands to run at specified times, such as a daily cleanup or report. The schedule is defined in a crontab.

Q. How would you troubleshoot a service that fails after reboot?

Answer: Check systemctl status, journalctl, unit dependencies, environment files, file permissions, network readiness, port conflicts and whether the service is enabled.

C. Networking and web fundamentals

Q. Explain the OSI model.

Answer: The OSI model is a seven-layer reference model: Physical, Data Link, Network, Transport, Session, Presentation and Application. It helps organise troubleshooting by layer.

Q. What is the difference between TCP and UDP?

Answer: TCP provides connection-oriented, reliable and ordered byte delivery. UDP sends datagrams without TCP's delivery and ordering guarantees.

Q. What happens when you enter a URL in a browser?

Answer: The browser checks caches, resolves DNS, establishes a connection, negotiates TLS for HTTPS, sends an HTTP request, receives the response and renders the page. The exact sequence can vary with caching and protocol features.

Q. What is DNS?

Answer: DNS is a distributed naming system that resolves names such as app.example.com into records such as IP addresses or aliases.

Q. What is the difference between A and CNAME?

Answer: A maps a hostname to an IPv4 address. CNAME aliases one hostname to another hostname.

Q. What is a reverse proxy?

Answer: A reverse proxy accepts client requests on behalf of backend servers and forwards them to the appropriate upstream service. Nginx and an ALB can perform reverse-proxy functions.

Q. What is a forward proxy?

Answer: A forward proxy represents clients making outbound requests, for example a corporate proxy that filters employee internet access.

Q. What is the difference between HTTP and HTTPS?

Answer: HTTPS is HTTP protected by TLS. TLS encrypts traffic and helps authenticate the server using certificates.

Q. What is a 502 error?

Answer: A gateway or proxy received an invalid response from an upstream server. Check upstream service status, port, protocol, proxy configuration and logs.

Q. What is a 504 error?

Answer: A gateway timed out waiting for an upstream response. Investigate network reachability, backend processing time, database latency and timeout settings.

Q. What is the difference between 401 and 403?

Answer: 401 means valid authentication is missing or not accepted. 403 means the request is understood but the authenticated caller is not permitted to perform it.

D. Git and version control

Q. What is Git?

Answer: Git is a distributed version-control system used to track source-code changes and collaborate across branches.

Q. What is a repository?

Answer: A repository is the project and its version history, including tracked files, commits, branches and tags.

Q. What is a branch?

Answer: A branch is a movable reference to a line of development. Developers use branches to work on features or fixes without changing the main line immediately.

Q. What is a commit?

Answer: A commit is a recorded snapshot of tracked changes with metadata and a message.

Q. What is a merge conflict?

Answer: A conflict occurs when Git cannot automatically combine changes, often because two branches changed the same lines. A developer must resolve and test the result.

Q. What is the difference between git pull and git fetch?

Answer: git fetch downloads remote updates without integrating them into the current branch. git pull fetches and then integrates, commonly by merge or rebase.

Q. What is .gitignore?

Answer: A file listing paths Git should normally ignore, such as node_modules, build output, local environment files and secrets. Already tracked files need separate untracking.

E. CI/CD and build tools

Q. What happens in a Jenkins pipeline?

Answer: A pipeline can check out code, install dependencies, build, run tests, scan, package an artifact and deploy it. The exact stages depend on the application and release policy.

Q. What is a Jenkinsfile?

Answer: A text file, usually stored in the repository, that defines a Jenkins pipeline as code.

Q. What is a build artifact?

Answer: A versioned output produced by a build, such as a ZIP file, container image or frontend dist directory.

Q. Why should builds be repeatable?

Answer: Repeatable builds make it easier to reproduce, audit, test and roll back releases. Pin versions and use lock files or controlled build images.

Q. What is an environment-specific configuration?

Answer: Settings that differ between development, testing and production, such as API endpoints, database hosts and feature flags. Secrets should be stored securely rather than committed to source.

F. Docker and containers

Q. What is Docker?

Answer: Docker is a platform and ecosystem for building, distributing and running container images. Containers package an application and its dependencies in an isolated process environment.

Q. What is a container image?

Answer: An immutable, layered package containing the filesystem and metadata needed to create containers.

Q. What is a container?

Answer: A running instance of an image, isolated using operating-system features such as namespaces and cgroups.

Q. What is a Dockerfile?

Answer: A text file containing instructions to build an image, such as FROM, WORKDIR, COPY, RUN, EXPOSE and CMD.

Q. What is the difference between an image and a container?

Answer: An image is the packaged template; a container is a created/running instance of that image.

Q. What is a Docker volume?

Answer: A Docker-managed or host-backed storage mechanism used to persist data beyond a container's lifecycle.

Q. What is port mapping?

Answer: It maps a host port to a container port, for example -p 8080:3000 makes container port 3000 reachable through host port 8080.

Q. How do you view running containers?

Answer: Use docker ps. Use docker ps -a to include stopped containers.

Q. How do you view container logs?

Answer: Use docker logs <container_name_or_id>; add -f to follow new output.

Q. How do you troubleshoot a container that exits?

Answer: Inspect docker ps -a, docker logs, docker inspect, exit code, environment, mounted files, permissions, command and resource limits.

Q. What is Docker Compose?

Answer: A tool for defining and running multi-container applications using a YAML configuration, such as frontend, backend and database services.

Q. Why should a database usually use persistent storage?

Answer: Container filesystems are disposable. Persistent volumes or managed database storage protect data across container replacement.

G. AWS and cloud

Q. What is cloud computing?

Answer: On-demand access to computing, storage, networking and managed services over a network, with usage-based pricing and elastic capacity.

Q. What is EC2?

Answer: Amazon Elastic Compute Cloud provides virtual servers called instances.

Q. What is an AMI?

Answer: An Amazon Machine Image is a template used to launch EC2 instances, including an operating system and configured software.

Q. What is a VPC?

Answer: A Virtual Private Cloud is a logically isolated network in AWS where you define IP ranges, subnets, routing and network controls.

Q. What is a subnet?

Answer: A subnet is a range of IP addresses within a VPC, associated with an Availability Zone and a route table.

Q. What makes a subnet public?

Answer: A subnet is commonly considered public when its route table has a route to an internet gateway. Resources also need appropriate public addressing and security rules to communicate with the internet.

Q. What is an Internet Gateway?

Answer: A VPC component that enables communication between a VPC and the internet when routing and addressing are configured.

Q. What is a NAT Gateway?

Answer: A managed service that lets resources in private subnets initiate outbound IPv4 internet connections without accepting unsolicited inbound internet connections.

Q. What is a Security Group?

Answer: A stateful virtual firewall attached to supported AWS resources. It controls allowed inbound and outbound traffic.

Q. What is a Network ACL?

Answer: A stateless subnet-level network filter. Return traffic must be allowed by the relevant rules.

Q. What is IAM?

Answer: AWS Identity and Access Management controls identities and permissions for AWS resources and actions.

Q. What is an IAM role?

Answer: An identity with permissions that can be assumed by a trusted principal, such as an EC2 instance or service.

Q. What is an Availability Zone?

Answer: A distinct location within an AWS Region designed to provide isolation from failures in other zones.

Q. What is an AWS Region?

Answer: A geographic area containing multiple Availability Zones.

Q. What is an Application Load Balancer?

Answer: A managed Layer 7 load balancer for HTTP/HTTPS that supports host/path routing, listeners, target groups and health checks.

Q. What is an ALB target group?

Answer: A collection of registered targets, with protocol, port and health-check settings.

Q. What is an ALB health check?

Answer: A periodic check against a target to determine whether it should receive traffic.

Q. What is Route 53?

Answer: AWS's DNS and domain registration service. It can host public/private DNS zones and route DNS queries.

Q. What is CloudWatch?

Answer: AWS monitoring and observability service for metrics, logs, alarms and dashboards.

Q. What is S3?

Answer: Amazon Simple Storage Service is object storage commonly used for artifacts, backups, static assets and logs.

Q. What is Auto Scaling?

Answer: A capability that adjusts the number of compute instances based on policies, schedules or health conditions.

Q. How would you secure an EC2 instance?

Answer: Use least-privilege IAM roles, private subnets where appropriate, restrictive security groups, patching, secure access such as SSM, encrypted storage and monitoring.

H. Kubernetes and orchestration

Q. What is Kubernetes?

Answer: An open-source container orchestration platform that automates deployment, scaling and management of containerised applications.

Q. What is a Pod?

Answer: The smallest deployable unit in Kubernetes. It contains one or more tightly coupled containers sharing network and storage context.

Q. What is a Deployment?

Answer: A Kubernetes controller that manages replicated Pods and rolling updates for stateless workloads.

Q. What is a Service?

Answer: A stable network abstraction that exposes a set of Pods, typically selected by labels.

Q. What is a ConfigMap?

Answer: A Kubernetes object for non-sensitive configuration data.

Q. What is a Secret?

Answer: A Kubernetes object intended for sensitive values. Access must still be controlled; do not assume a Secret is automatically safe without encryption/access controls.

Q. What is a Namespace?

Answer: A logical grouping boundary for namespaced Kubernetes resources within a cluster.

Q. What is a Node?

Answer: A worker machine, virtual or physical, that runs Pods.

Q. What is a readiness probe?

Answer: A check that determines whether a container is ready to receive traffic.

Q. What is a liveness probe?

Answer: A check used to determine whether a container should be restarted.

Q. What is a rolling update?

Answer: Gradually replacing old Pods with new Pods while maintaining application availability within configured limits.

Q. What is the difference between Docker and Kubernetes?

Answer: Docker is commonly used to build and run containers; Kubernetes orchestrates containerised workloads across a cluster. They solve related but different problems.

I. Monitoring, logging and incident handling

Q. What is monitoring?

Answer: Monitoring collects and evaluates metrics, logs and traces to understand availability, performance and system health.

Q. What is the difference between logs and metrics?

Answer: Logs are event records with details. Metrics are numerical measurements over time, such as CPU percentage or request rate.

Q. What is tracing?

Answer: Distributed tracing follows a request across multiple services and records timing and relationships between operations.

Q. What is an SLA?

Answer: A Service Level Agreement is a formal commitment about service levels, such as availability or support response.

Q. What is an SLI?

Answer: A Service Level Indicator is a measurable signal of service performance, such as successful request percentage or latency.

Q. What is an SLO?

Answer: A Service Level Objective is a target for an SLI over a defined period.

Q. What is an alert threshold?

Answer: A condition that triggers an alert, such as error rate above a defined value for a sustained period.

Q. How do you investigate high CPU?

Answer: Confirm the metric and time window, identify processes using top/ps, check workload and recent changes, inspect application logs and compare with traffic. Avoid restarting before collecting evidence.

Q. How do you investigate disk full?

Answer: Use df -h and du -xhd1 on the affected filesystem, identify large logs/dumps/temp files, verify whether files are actively used, then clean or rotate safely and address the root cause.

Q. How do you handle a production incident?

Answer: Assess impact, communicate, stabilise service safely, collect evidence, identify the failing component, apply a controlled fix or rollback, validate recovery and document root cause and preventive actions.

J. Scenario-based questions interviewers commonly ask

Q. Website is down. What do you check first?

Answer: Confirm the exact URL, time, source network and error. Check DNS resolution, ALB listener and target health, security groups/routes, Nginx status/logs, backend service and database connectivity. Follow the request path and find the first failing hop.

Q. ALB targets are unhealthy, but Nginx is running. Why?

Answer: The configured health-check path may be wrong, the target port may be incorrect, the response code may not match the success matcher, or ALB-to-instance security-group access may be blocked. Test the exact health-check URL locally and from an allowed source.

Q. The website loads, but login fails. What do you check?

Answer: Inspect browser network requests, API URL, response status, CORS if cross-origin, authentication middleware, backend logs, database availability and any identity-provider dependency.

Q. The backend works locally but not through Nginx. What could be wrong?

Answer: Check Nginx location matching, proxy_pass URI behaviour, upstream host/port, security groups, backend bind address, headers and Nginx error logs.

Q. The domain resolves but the website is unavailable. Explain.

Answer: DNS only resolves the name. The ALB may be unhealthy or misconfigured, listeners/certificate may be wrong, network access may be blocked or the frontend application may be down.

Q. A deployment introduced errors. What is your response?

Answer: Stop further rollout, assess impact, preserve logs/metrics, compare the release with the previous version, roll back using the release plan if necessary, verify service and data integrity, then perform root-cause analysis.

Q. A database connection suddenly fails. What do you check?

Answer: Check DB service and port, private DNS resolution, routes, security-group source/destination, host firewall, DB bind/listen configuration, credentials, allowed database user host and connection limits.

Q. How do you explain your project in an interview?

Answer: Describe the user request path: DNS resolves the domain to the ALB; HTTPS reaches the ALB listener; the ALB forwards to healthy frontend EC2 targets; Nginx serves the page and proxies API requests; Node.js validates and processes them; MySQL stores data over private networking. Then explain how you deployed, secured, tested and monitored each layer.

21. Separate Practical Runbook – Create and Deploy a Web Page Step by Step

This final section is intentionally a standalone procedure. Follow it from an empty project directory to a working page served by Nginx, then connect it to your backend and AWS ALB.

This procedure creates a simple Expense Management web page using HTML, CSS and JavaScript. It then deploys the static page to an Nginx frontend EC2 instance. The page is deliberately simple so that you understand every file and deployment step before adding a framework.

Part 1 – Prepare the Linux frontend server

111. Sign in to the AWS console and open EC2.
112. Choose Launch instance and give it a clear name such as expense-frontend-01.
113. Choose a supported Amazon Linux AMI and a suitable lab instance type.
114. Select the project VPC and a frontend subnet. For an ALB-based design, keep the instance reachable from the ALB through private networking; do not expose unnecessary ports publicly.
115. Attach the Frontend security group. Allow inbound TCP 80 only from the ALB security group. Allow administrator access through Systems Manager Session Manager or a restricted SSH rule if required.
116. Launch the instance and wait until EC2 status checks pass.
117. Connect using Session Manager or your approved SSH method.
118. Update the operating system packages using the command appropriate to the installed Amazon Linux version.
119. Install Nginx using the package manager supported by the operating system.
120. Enable and start Nginx, then check that its service state is active.
121. Confirm Nginx is listening on TCP port 80 using ss -lntp.
122. Test the default page locally using curl -I http://127.0.0.1.

```
# Identify operating system
cat /etc/os-release

# Update packages on Amazon Linux 2023
sudo dnf update -y

# Install and start Nginx
sudo dnf install -y nginx
sudo systemctl enable --now nginx
sudo systemctl status nginx

# Verify port and local response
sudo ss -lntp | grep ':80'
curl -I http://127.0.0.1
```

Part 2 – Create the website directory and files

123. Create a project directory in your home directory, for example ~/expense-web.
124. Inside it, create three files: index.html, style.css and app.js.
125. Use index.html for the page structure and form fields.
126. Use style.css for colours, spacing, typography and responsive layout.
127. Use app.js for browser-side interactions such as validating the form and showing a confirmation message.
128. Save all three files and verify their names and contents with ls -l and cat.
129. Do not place database passwords, admin tokens or private keys in HTML or JavaScript. Browser code is visible to every visitor.

```
mkdir -p ~/expense-web
cd ~/expense-web
touch index.html style.css app.js
ls -l
```

Part 3 – Add the HTML page

HTML means HyperText Markup Language. It defines the structure and content of the page. The following is a complete minimal example for an expense entry form.

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Expense Management</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header class="site-header">
    <h1>Expense Management</h1>
    <p>Record and review your daily expenses</p>
  </header>

  <main class="container">
    <section class="card">
      <h2>Add a new expense</h2>

      <form id="expense-form">
        <label for="description">Description</label>
        <input id="description" name="description"
               type="text" maxlength="100" required>

        <label for="amount">Amount (INR)</label>
        <input id="amount" name="amount"
               type="number" min="0.01" step="0.01" required>

        <label for="category">Category</label>
        <select id="category" name="category" required>
          <option value="">Select category</option>
          <option value="Food">Food</option>
          <option value="Travel">Travel</option>
          <option value="Bills">Bills</option>
          <option value="Other">Other</option>
        </select>

        <label for="expense-date">Date</label>
        <input id="expense-date" name="date" type="date" required>

        <button type="submit">Save expense</button>
      </form>

      <p id="message" role="status" aria-live="polite"></p>
    </section>
  </main>

  <script src="app.js"></script>
</body>
</html>
```

Part 4 – Add CSS styling

CSS means Cascading Style Sheets. It controls how HTML elements look. Save the following in style.css.

```
* {
```

```
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: Arial, sans-serif;
  background: #f3f6fa;
  color: #1d2939;
}

.site-header {
  background: #12304a;
  color: white;
  padding: 28px 20px;
  text-align: center;
}

.site-header h1 {
  margin: 0 0 8px;
}

.site-header p {
  margin: 0;
}

.container {
  width: min(92%, 620px);
  margin: 32px auto;
}

.card {
  background: white;
  padding: 24px;
  border-radius: 12px;
  box-shadow: 0 4px 18px rgba(0,0,0,.08);
}

form {
  display: grid;
  gap: 10px;
}

label {
  font-weight: 600;
  margin-top: 8px;
}

input, select {
  width: 100%;
  padding: 11px;
  border: 1px solid #cbd5e1;
  border-radius: 6px;
  font: inherit;
}

button {
  margin-top: 14px;
  padding: 12px;
```

```
  border: 0;
  border-radius: 6px;
  background: #176b87;
  color: white;
  font-size: 1rem;
  cursor: pointer;
}

button:hover {
  background: #12566d;
}

#message {
  margin-top: 14px;
  font-weight: 600;
}

@media (max-width: 480px) {
  .card {
    padding: 16px;
  }
}
```

Part 5 – Add JavaScript behaviour

JavaScript adds behaviour in the browser. This example validates the form and displays a message. It does not save anything permanently yet; the next part connects the page to the backend API.

```
const form = document.getElementById("expense-form");
const message = document.getElementById("message");

form.addEventListener("submit", function (event) {
  event.preventDefault();

  const description =
    document.getElementById("description").value.trim();
  const amount =
    Number(document.getElementById("amount").value);
  const category =
    document.getElementById("category").value;
  const date =
    document.getElementById("expense-date").value;

  if (!description || !category || !date ||
      !Number.isFinite(amount) || amount <= 0) {
    message.textContent = "Please enter valid expense details.";
    return;
  }

  message.textContent =
    `Expense details captured: ${description}, ₹${amount.toFixed(2)}. ` +
    "Backend connection is the next step.";

  form.reset();
});
```

Part 6 – Test the page locally before deployment

130. Check that all three files exist and that index.html links to style.css and app.js using the correct relative paths.

131. Open index.html in a browser if you are working on a local computer, or use a temporary local web server for testing.
132. Check that the page layout is readable on desktop and mobile widths.
133. Enter an empty form and verify browser validation prevents submission.
134. Enter a valid description, amount, category and date; verify the confirmation message appears.
135. Open browser developer tools and check the Console tab for JavaScript errors.
136. Check the Network tab to confirm style.css and app.js load successfully with HTTP 200.

Part 7 – Copy the web page to Nginx document root

137. Create the Nginx document root if it does not exist.
138. Copy index.html, style.css and app.js into the document root.
139. Set directories to readable/searchable permissions and files to readable permissions.
140. Keep the files owned by root or the deployment owner; Nginx only needs read access for static files.
141. Verify the copied files are present and readable.

```
sudo mkdir -p /usr/share/nginx/html
sudo cp ~/expense-web/index.html /usr/share/nginx/html/
sudo cp ~/expense-web/style.css /usr/share/nginx/html/
sudo cp ~/expense-web/app.js /usr/share/nginx/html/

sudo chown -R root:root /usr/share/nginx/html
sudo find /usr/share/nginx/html -type d -exec chmod 755 {} \\;
sudo find /usr/share/nginx/html -type f -exec chmod 644 {} \\;

ls -l /usr/share/nginx/html
```

Part 8 – Configure Nginx to serve the page

142. Create a dedicated Nginx server configuration file in /etc/nginx/conf.d/.
143. Set the server to listen on port 80.
144. Set server_name to the domain you intend to use, or use a suitable default server configuration for the lab.
145. Set root to /usr/share/nginx/html and index to index.html.
146. Use try_files so the requested file is served and unknown frontend routes can fall back to index.html if the application is a single-page app.
147. Test the configuration before reloading Nginx.
148. Reload Nginx and check its service status.

```
sudo tee /etc/nginx/conf.d/expense.conf > /dev/null <<'EOF'
server {
    listen 80;
    server_name <DOMAIN> www.<DOMAIN>;

    root /usr/share/nginx/html;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }
}
EOF

sudo nginx -t
sudo systemctl reload nginx
sudo systemctl status nginx
```

Replace <DOMAIN> with your actual domain. If DNS is not ready yet, test using the instance's private IP from an allowed network path, or use a temporary default server block for the lab.

Part 9 – Test the page through Nginx

149. Run curl -I http://127.0.0.1 on the frontend server.

150. Run curl http://127.0.0.1 and confirm the HTML is returned.
151. From a machine allowed by the frontend security group, test the frontend private IP on port 80.
152. Open the URL in a browser and confirm the page displays correctly.
153. Use browser developer tools to verify that CSS and JavaScript load.
154. If the page fails, check Nginx configuration, service status, file permissions and /var/log/nginx/error.log.

```
curl -I http://127.0.0.1
curl -s http://127.0.0.1 | head
sudo tail -n 50 /var/log/nginx/access.log
sudo tail -n 50 /var/log/nginx/error.log
```

Part 10 – Connect the web page to the backend API

The earlier JavaScript example only shows a local confirmation. To save an expense, the browser must send a request to a backend API. First confirm the real API route, method, request fields and authentication requirements from your backend source code or API documentation.

For illustration only, suppose the backend supports POST /api/expenses and expects description, amount, category and date. Replace the route and field names if your actual backend differs.

```
// Example only – adapt endpoint and payload to your API
const form = document.getElementById("expense-form");
const message = document.getElementById("message");

form.addEventListener("submit", async (event) => {
  event.preventDefault();

  const payload = {
    description: document.getElementById("description").value.trim(),
    amount: Number(document.getElementById("amount").value),
    category: document.getElementById("category").value,
    date: document.getElementById("expense-date").value
  };

  if (!payload.description ||
      !Number.isFinite(payload.amount) ||
      payload.amount <= 0 ||
      !payload.category ||
      !payload.date) {
    message.textContent = "Enter valid expense details.";
    return;
  }

  try {
    const response = await fetch("/api/expenses", {
      method: "POST",
      headers: {
        "Content-Type": "application/json"
        // Add the required Authorization header
        // only if your API requires it.
      },
      body: JSON.stringify(payload)
    });

    const result = await response.json().catch(() => ({}));

    if (!response.ok) {
      throw new Error(result.message || `Request failed: ${response.status}`);
    }
```

```
    message.textContent = "Expense saved successfully.";
    form.reset();
  } catch (error) {
    message.textContent = `Could not save expense: ${error.message}`;
  }
});
```

Using a relative URL such as /api/expenses means the browser sends the request to the same public origin that served the page. Nginx can then proxy /api/ to the backend. Do not put database credentials in this JavaScript; the browser must never connect directly to MySQL.

Part 11 – Configure Nginx reverse proxy for the API

155. Confirm the backend service is running on its private IP and intended port.
156. From the frontend host, test connectivity to the backend port.
157. Confirm the exact API route works when called directly from the frontend host.
158. Add a location /api/ block to the Nginx server configuration.
159. Set proxy_pass to the backend private IP and port.
160. Forward the Host, client IP and original scheme headers as appropriate.
161. Check the trailing-slash behaviour of proxy_pass so the API path is not accidentally changed.
162. Run nginx -t and reload Nginx.
163. Call the API through the frontend URL and inspect the backend logs.

```
# Connectivity test from frontend to backend
nc -vz <BACKEND_PRIVATE_IP> 8080

# Example Nginx server configuration
server {
    listen 80;
    server_name <DOMAIN> www.<DOMAIN>;

    root /usr/share/nginx/html;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }

    location /api/ {
        proxy_pass http://<BACKEND_PRIVATE_IP>:8080;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}

sudo nginx -t
sudo systemctl reload nginx
```

Part 12 – Put two frontend servers behind an ALB

164. Create a second frontend EC2 instance in a different Availability Zone where possible.
165. Repeat the Nginx installation, frontend file deployment and configuration on the second instance.
166. Ensure both frontend instances use the same tested application version and backend destination.
167. Create an EC2 target group using HTTP port 80 and register both frontend instances.
168. Configure a health-check path that actually returns a successful status from both frontend instances.
169. Create an internet-facing Application Load Balancer across at least two public subnets in different Availability Zones.
170. Attach an ALB security group that permits intended inbound HTTPS traffic.

171. Configure the frontend security group to permit HTTP only from the ALB security group.
172. Create an HTTPS listener on port 443 and attach a valid ACM certificate for the domain.
173. Forward HTTPS traffic to the frontend target group.
174. Optionally configure HTTP port 80 to redirect to HTTPS port 443.
175. Wait until both target instances show Healthy before proceeding.

Part 13 – Connect DNS and test the public website

176. Confirm that your domain is registered and that you control its DNS delegation.
177. Create or verify the public hosted zone in Route 53.
178. At the registrar, set the authoritative name servers to the name servers assigned to the Route 53 hosted zone.
179. Wait for delegation/caches to update and verify using dig NS <DOMAIN>.
180. Create a Route 53 Alias A record for the root domain pointing to the ALB.
181. Create an appropriate Alias record for www if that hostname is required.
182. Open the domain using HTTPS and verify the certificate matches the hostname.
183. Submit a test expense and confirm the backend receives the API request.
184. Verify that the backend writes the expected record to MySQL.
185. Check the ALB target health, Nginx access/error logs and backend service logs.

Part 14 – Final validation and troubleshooting checklist

Test	Expected result	If it fails, inspect
DNS lookup	Domain resolves to ALB destination	Registrar delegation, hosted zone, record, TTL
HTTPS connection	Certificate valid; page opens	ACM certificate, listener, hostname, ALB security group
ALB target health	Both frontend targets healthy	Health path, target port, Nginx, frontend SG
Static assets	CSS and JS return successfully	HTML file paths, Nginx root, file permissions
API request	Expected success status and JSON	Browser Network tab, Nginx proxy, backend route
Database write	Test expense row is present	Backend logs, DB connectivity, schema, grants
Failure tolerance	Remaining healthy target serves page	Target health, capacity, AZ and ALB configuration

Part 15 – What you should be able to explain after completing the lab

- How HTML creates the page structure, CSS styles it and JavaScript handles browser interactions.
- How Nginx serves static files from its document root.
- How the browser sends an API request using fetch().
- How Nginx reverse-proxies /api/ requests to the private backend.
- How the backend validates the request and stores the data in MySQL.
- How DNS resolves the website domain to the ALB.
- How the ALB listener forwards traffic to healthy frontend targets.
- How security groups restrict traffic between the ALB, frontend, backend and database.
- How to use curl, dig, ss, systemctl and logs to isolate a failure.

End of standalone web-page deployment runbook. Keep this section as your execution checklist. The earlier chapters explain the concepts; this final section is the ordered hands-on procedure.